

1. PROJECT OVERVIEW

- **Project Title:** ComicCraft - AI Comic Story Creator using Gemini Models

2. ARCHITECTURAL ORGANIZATION To ensure scalability and ease of debugging, the ComicCraft codebase is structured using a strict Model-View-Controller (MVC) paradigm. The project directory is clearly segregated into source code, visual assets, and UI templates:

- `app/`: The core backend directory housing the Python application.
 - `main.py`: The application entry point that initializes the FastAPI instance.
 - `main_routes.py`: The controller file that defines HTTP endpoints and manages the sequence of AI operations.
 - `gemini_handler.py`: Encapsulates all API communication with Google's Gemini LLMs for story structuring and dialogue.
 - `image_handler.py`: Dedicated module for querying the Hugging Face Stable Diffusion API, complete with exception-handling mechanisms.
 - `layout_manager.py`: Responsible for mapping generated narrative text to the corresponding visual panel assets.
 - `pdf_compiler.py`: Contains the FPDF logic to compile layouts into downloadable multi-page documents.
- `templates/`: Houses the Jinja2 HTML files (`index.html`, `preview.html`, `export_success.html`).
- `static/`: The repository for CSS stylesheets, background graphics, AI-generated panels (`static/panels/`), and final PDF outputs (`static/exports/`).

3. CODE READABILITY & STANDARDS

- **PEP 8 Alignment:** All backend scripts follow Python PEP 8 conventions. Variable names are highly descriptive (e.g., `panel_outline`, `image_paths`), and logic blocks are consistently indented.
- **Functional Segregation:** The application avoids large, monolithic scripts. Each file is dedicated to a single operational task, making the codebase easy to read and maintain.
- **Inline Documentation:** Critical sections of the code, especially API exception handling and PDF coordinate calculations, are extensively commented to aid external reviewers in understanding the system's logic.

4. MODULARITY AND REUSABILITY

- **Dynamic AI Prompting:** The `gemini_handler.py` and `image_handler.py` modules are designed to accept dynamic parameters. This means the exact same functions can be reused for any comic genre without requiring code alterations.
- **Agnostic Layout Generation:** The layout dictionaries created by the backend are length-agnostic. Whether generating a 3-panel strip or an 8-panel story, the `pdf_compiler.py` can reuse the same loop logic to format the document seamlessly.
- **Credential Isolation:** The `.env` structure ensures that the codebase remains portable and can be reused on any host machine simply by swapping the API keys, without modifying the underlying Python scripts.